

ESPRIT School of Engineering

PROJET DE FIN D'ÉTUDES

Diplôme National d'Ingénieur – Spécialité TWIN

Plateforme AIOps

Analyse Intelligente des Pipelines CI/CD

Réalisé par : **Khalil Ben Yahia Wenich**

Encadrante ESPRIT : Mme Houda Jendoubi

Organisme d'accueil : Capgemini – Altran Telnet Corporation Tunisie

Année académique 2025 – 2026

Dédicace

À mes chers parents, pour leur soutien indéfectible et leurs sacrifices,

À ma famille et mes amis, pour leurs encouragements constants,

À toutes les personnes qui ont contribué, directement ou indirectement, à l'aboutissement de ce travail.

Je dédie ce modeste accomplissement.

i

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à sa réussite.

Je remercie en premier lieu mon encadrante académique, Mme Houda Jendoubi, pour ses orientations, ses précieux conseils et sa disponibilité tout au long de ce travail.

J'adresse également mes sincères remerciements à mon encadrant entreprise chez Capgemini – Altran Telnet Corporation Tunisie, ainsi qu'à toute l'équipe d'ingénierie, pour leur accueil, leur confiance et le partage de leur expertise en matière de pratiques CI/CD et d'outillage DevOps de niveau production.

Mes remerciements vont aussi au corps enseignant et au personnel administratif de l'ESPRIT School of Engineering pour la qualité de la formation dispensée tout au long de mes études.

Enfin, je remercie les membres du jury d'avoir accepté d'évaluer ce travail.

ii

Résumé

Ce projet de fin d'études, réalisé au sein de Capgemini – Altran Telnet Corporation Tunisie, consiste en la conception et le développement d'une plateforme AIOps (Artificial Intelligence for IT Operations) pour l'analyse automatisée des pipelines CI/CD.

La solution reçoit les événements GitLab en temps réel via des webhooks sécurisés, collecte les rapports de qualité (SonarQube) et de sécurité (Trivy), puis soumet l'ensemble à un grand modèle de langage (Groq – Llama 3.3 70B) pour produire une analyse de cause racine, un score de risque et des suggestions de correction priorisées — le tout en moins de dix secondes. La plateforme intègre également la gestion des incidents, une base de connaissances auto-alimentée, un score de santé par projet (grade A à F) et des rapports hebdomadaires agrégés, présentés dans un tableau de bord React interactif en temps réel. Elle est conteneurisée avec Docker, déployée sur Kubernetes (k3s) via Helm, et dotée d'un pipeline CI/CD Jenkins complet avec tests automatisés.

Mots-clés : AIOps, CI/CD, Analyse de cause racine, LLM, GitLab, Kubernetes, Jenkins, Node.js, React.

Abstract

This final year project, carried out at Capgemini – Altran Telnet Corporation Tunisia, consists of designing and developing an AIOps platform for the automated analysis of CI/CD pipelines. The solution receives GitLab events in real time through secure webhooks, collects quality (SonarQube) and security (Trivy) reports, then submits the aggregated context to a Large Language Model (Groq – Llama 3.3 70B) to produce a root-cause analysis, a risk score and prioritized fix suggestions — all within ten seconds. The platform also provides incident management, a self-feeding knowledge base, a per-project health score (grade A to F) and weekly aggregated reports, presented in a real-time React dashboard. It is containerized with Docker, deployed on Kubernetes (k3s) through Helm, and equipped with a complete Jenkins CI/CD pipeline featuring automated tests.

Keywords: AIOps, CI/CD, Root-Cause Analysis, LLM, GitLab, Kubernetes, Jenkins, Node.js, React.

iii

Table des matières

Dédicace	i
Remerciements	ii
Résumé / Abstract	iii
Liste des abréviations	x
Introduction générale	1
1 Contexte général et problématique	2
1.1 Introduction	2
1.2 Organisme d'accueil	2
1.2.1 Positionnement organisationnel	2
1.2.2 Analyse des parties prenantes	3
1.2.3 Analyse SWOT de l'initiative	5
1.3 Étude de l'existant	5
1.4 Critique de l'existant	6
1.5 Solution proposée	7
1.6 Méthodologie de travail	8
1.7 Conclusion	9
2 État de l'art et étude comparative	10
2.1 Introduction	10
2.2 Plateformes AIOps et d'observabilité commerciales	10
2.2.1 Datadog et Dynatrace	10
2.2.2 Moogsoft et BigPanda	10
2.2.3 PagerDuty / Jeli et outils de gestion d'incidents	11
2.3 Outillage natif CI/CD	11
2.4 État de l'art académique : LLMs pour l'analyse de logs et RCA	11

2.5 Synthèse comparative	12
2.6 Positionnement de la contribution	12
2.7 Conclusion	13
3 Analyse et spécification des besoins	14
3.1 Introduction	14
3.2 Identification des acteurs	14
3.3 Besoins fonctionnels	14
3.3.1 Tableau de bord	14
3.3.2 Gestion des pipelines	15
3.3.3 Analyse IA – Analyse de cause racine	15
3.3.4 Sécurité et qualité du code	15
3.3.5 Base de connaissances	15
3.3.6 Score de santé, incidents et rapports	16
3.4 Besoins non fonctionnels	16
3.5 Comportement fonctionnel : diagramme d'activité	17
3.6 Modélisation des cas d'utilisation	18
3.6.1 Raffinement des cas d'utilisation	19
3.7 Matrice de traçabilité des besoins	23
3.8 Conclusion	24
4 Conception et architecture	25
4.1 Introduction	25
4.2 Architecture globale	25
4.2.1 Diagramme de composants	26
4.2.2 Diagramme de déploiement physique	26
4.3 Architecture backend	27
4.4 Flux de traitement principal	27
4.5 Modèles de données	28

4.5.1 Diagramme de classes de la couche service	31
4.6 Services métier	32
4.6.1 Service de score de santé	32
4.6.2 Service de base de connaissances	32
4.6.3 Autres services	32
4.7 Conception de l'API REST	33
4.7.1 Justification des choix de conception de l'API	33
4.7.2 Justification des choix de conception de la base de données . . .	34
4.8 Vues comportementales complémentaires	34
4.9 Architecture frontend	35
4.10 Conclusion	36
5 Implémentation, tests et déploiement	37
5.1 Introduction	37
5.2 Environnement de travail	37
5.2.1 Environnement matériel	37
5.2.2 Environnement logiciel	37
5.3 Stack technologique	38
5.4 Interfaces de l'application	40
5.5 Extraits de code clés	41
5.5.1 Validation de la signature webhook	41
5.5.2 Calcul pondéré du score de santé	42
5.5.3 Worker BullMQ avec concurrence bornée et relances	42
5.5.4 Connexion MongoDB résiliente	43
5.5.5 Récupération de données en temps réel côté frontend	43
5.6 Tests automatisés	44
5.6.1 Stratégie de test et pyramide des tests	45
5.6.2 Ce qui n'est intentionnellement pas couvert	45
5.7 Conteneurisation	46
5.8 Déploiement Kubernetes avec Helm	47

5.9 Pipeline CI/CD Jenkins	48
5.10 Défis d'ingénierie et solutions	49
5.11 Sécurité	51
5.12 Résultats	51
5.12.1 Métriques de performance	52
5.12.2 Validation de bout en bout	52
5.12.3 Fonctionnalités validées	52
5.13 Conclusion	53
6 Discussion critique, limites et mitigations	54
6.1 Introduction	54
6.2 Limites méthodologiques	54
6.2.1 Précision de classification IA mesurée sur un échantillon limité	54
6.2.2 Absence d'étude d'utilisabilité formelle	54
6.3 Limites architecturales	55
6.3.1 Réplica unique, pas de haute disponibilité	55
6.3.2 Instance MongoDB unique, sans jeu de réplicas ni politique de sauvegarde	55
6.3.3 Gestion des secrets sans coffre-fort	55
6.4 Confidentialité des données et dépendance au LLM tiers	55
6.5 Limites opérationnelles et de processus	56
6.6 Questions de jury anticipées	56
6.7 Conclusion	57
7 Gestion du projet	59
7.1 Introduction	59
7.2 Structure de découpage du travail (WBS)	59
7.3 Registre des risques	59
7.4 Répartition de l'effort	60
7.5 Conclusion	61
Conclusion générale et perspectives	62

Bibliographie	63
Annexes	63
A Fichier Helm Values complet	64
B Jenkinsfile complet	66
C Extraits de code supplémentaires	68
D Glossaire	70

Liste des figures

1.1 Positionnement organisationnel simplifié du stage	3
1.2 Diagramme de Gantt du projet (année académique 2025–2026)	8
3.1 Diagramme d'activité du worker d'analyse de pipeline, avec ses deux branches conditionnelles	18
3.2 Diagramme de cas d'utilisation global avec relations de généralisation, inclusion et extension	19
4.1 Architecture globale de la plateforme AIOps	25
4.2 Diagramme de composants UML avec interfaces fournies (plein) et requises (tirets)	26
4.3 Diagramme de déploiement physique de l'environnement de lab	26
4.4 Diagramme de séquence : analyse de pipeline	28
4.5 Diagramme de classes UML complet de la couche persistance (7 collections Mongoose + 4 objets-valeurs embarqués)	30
4.6 Diagramme de classes de la couche service et ses dépendances sur la couche persistance	31
4.7 Diagramme de séquence : commentaire automatique de risque sur MR	35
4.8 Diagramme de séquence : résolution d'incident et génération de post-mortem IA	35
5.1 Interface – Tableau de bord principal	40
5.2 Interface – Détail d'une analyse IA	41
5.3 Interface – Panneau sécurité (vulnérabilités)	41
5.4 Pyramide des tests : couverture implémentée vs extensions planifiées	45
5.5 Architecture de déploiement Kubernetes (k3s)	47
5.6 Flux CI/CD de bout en bout	49
7.1 Structure de découpage du travail (WBS)	59

v

Liste des tableaux

1.1 Analyse des parties prenantes	4
1.2 Analyse SWOT	5
1.3 Décomposition du temps de diagnostic manuel (moyenne de 5 incidents observés)	7
1.4 Objectifs du projet	8
2.1 Synthèse comparative des approches existantes	12
3.1 Besoins non fonctionnels avec critères d'acceptation mesurables	17
3.2 Description du cas d'utilisation : « Analyser un pipeline en échec »	20
3.3 Description du cas d'utilisation : « Résoudre une analyse IA »	21
3.4 Description du cas d'utilisation : « Gérer un incident et générer un post-mortem »	22
3.5 Description du cas d'utilisation : « Commenter une MR avec évaluation de risque »	23
3.6 Matrice de traçabilité : objectifs → fonctionnalités → cas d'utilisation	24
4.1 Étapes de traitement de l'analyse de pipeline	28
4.2 Collections principales de la base de données	29
4.3 Métriques et pondérations du score de santé	32
4.4 Correspondance score–grade	32
4.5 Principaux endpoints de l'API REST	33
5.1 Stack backend	38
5.2 Stack frontend	39
5.3 Stack infrastructure et DevOps	40
5.4 Suite de tests automatisés	44
5.5 Lacunes de couverture et mitigation manuelle	46

5.6 Problèmes d'infrastructure rencontrés et leurs résolutions	50
5.7 Mesures de sécurité	51
5.8 Résultats de performance	52
6.1 Questions de jury anticipées et réponses préparées	57
7.1 Registre des risques	60
7.2 Répartition approximative de l'effort	61

vi

Liste des abréviations

AIOps Artificial Intelligence for IT Operations

CI/CD Intégration Continue / Déploiement Continu

LLM Large Language Model (Grand modèle de langage)

RBAC Role-Based Access Control (Contrôle d'accès basé sur les rôles)

JWT JSON Web Token

HMAC Hash-based Message Authentication Code

MTTR Mean Time To Resolve (Délai moyen de résolution)

CVE Common Vulnerabilities and Exposures

MR Merge Request (Demande de fusion)

API Application Programming Interface

ODM Object Document Mapper

SPA Single Page Application

PVC Persistent Volume Claim

k3s Distribution Kubernetes légère

KB Knowledge Base (Base de connaissances)

WBS Work Breakdown Structure (Structure de découpage du travail)

x

Introduction générale

L'ingénierie logicielle moderne repose massivement sur les méthodologies DevOps, qui rapprochent les équipes de développement et d'exploitation pour livrer plus vite et avec plus de fiabilité. Au cœur de cette approche, les pipelines d'Intégration Continue et de Déploiement Continu (CI/CD) automatisent la construction, les tests et la mise en production des logiciels. Cette automatisation présente toutefois un revers : elle produit une quantité considérable de données techniques — logs d'exécution, rapports de qualité de code, analyses de vulnérabilités — dont l'exploitation manuelle devient rapidement ingérable.

Lorsqu'un pipeline échoue, l'ingénieur doit parcourir des milliers de lignes de logs, croiser les résultats de plusieurs outils hétérogènes et identifier seul la cause racine. Ce diagnostic prend en moyenne 30 à 90 minutes par incident, souvent pour des problèmes récurrents qui avaient déjà été résolus dans le passé, mais dont la solution n'a jamais été capitalisée.

C'est dans ce contexte que s'inscrit le présent projet de fin d'études, qui propose une plateforme AIOps appliquant l'intelligence artificielle à la supervision des pipelines CI/CD. L'objectif est d'automatiser l'analyse, de centraliser les sources de données, de fournir un diagnostic de cause racine assisté par IA, et de capitaliser les solutions récurrentes.

Ce rapport s'articule en sept chapitres. Le premier chapitre présente le contexte général, l'organisme d'accueil, l'étude de l'existant et la solution proposée. Le deuxième chapitre dresse un état de l'art et positionne la contribution du projet par rapport aux travaux commerciaux et académiques existants. Le troisième chapitre porte sur l'analyse et la spécification des besoins fonctionnels et non fonctionnels. Le quatrième chapitre détaille la conception de la solution, son architecture, ses modèles de données, ses services et ses API. Le cinquième chapitre décrit l'implémentation, les tests, le déploiement (Docker, Helm, Kubernetes, Jenkins) et les résultats obtenus. Le sixième chapitre discute de manière critique les limites de la solution et les mitigations identifiées. Enfin, le septième chapitre couvre les aspects de gestion de projet (structure de découpage, registre des risques), avant la conclusion générale, les perspectives et les annexes.

CHAPITRE 1

Contexte général et problématique

1.1 Introduction

Ce chapitre pose le cadre du projet. Il présente l'organisme d'accueil, analyse la situation actuelle et ses limites, et présente la solution proposée ainsi que la méthodologie de travail adoptée.

1.2 Organisme d'accueil

Ce projet a été réalisé au sein de Capgemini – Altran Telnnet Corporation (ATC) Tunisie. ATC est une co-entreprise créée en 2008 entre le groupe Altran et le groupe tunisien Telnnet Holding. La société est spécialisée dans l'ingénierie et le développement de logiciels embarqués et de systèmes temps réel, notamment pour les secteurs défense, sécurité, aéronautique et automobile.

Suite à l'acquisition d'Altran par le groupe Capgemini en 2020, l'entité a progressivement rejoint la marque Capgemini Engineering, la ligne de métier dédiée aux services d'ingénierie et de R&D au sein du groupe Capgemini. Capgemini est aujourd'hui un leader mondial du conseil, des services technologiques et de la transformation numérique, présent dans plus de 50 pays. Localement, ATC accompagne plusieurs entreprises clientes internationales dans leur transformation DevOps et d'ingénierie de plateforme, où les équipes d'ingénierie font face quotidiennement à des pipelines CI/CD défaillants — précisément le point de douleur opérationnel adressé par ce projet.

1.2.1 Positionnement organisationnel

La figure 1.1 situe le stage au sein de l'organisation d'ingénierie de la société. Le projet a été accueilli par une unité DevOps/Ingénierie de Plateforme, elle-même intégrée dans un département de génie logiciel relevant de la direction Capgemini Engineering Tunisie. Cette unité est notamment responsable de la maintenance de l'outillage CI/CD (GitLab, Jenkins, SonarQube) utilisé par plusieurs équipes d'ingénierie orientées clients — ce qui en faisait le commanditaire naturel d'une initiative AIOps ciblant ces mêmes pipelines.

Capgemini Engineering Tunisie

(Direction pays)

Département Génie Logiciel

Unité DevOps / Équipes Delivery Client

Ingénierie de (embarqué, automobile, ...)

Plateforme

pipelines supervisés

Stagiaire PFE

(ce projet)

Figure 1.1 : Positionnement organisationnel simplifié du stage

1.2.2 Analyse des parties prenantes

Le tableau 1.1 identifie les parties prenantes du projet ainsi que leur intérêt et leur influence respectifs, ce qui a guidé les décisions de priorisation tout au long des sprints (par exemple, le tableau de bord et l'analyse de cause racine ont été priorisés par rapport au rapport hebdomadaire, car ils servent directement la partie prenante à la plus forte influence).

Chapitre 1. Contexte général et problématique Plateforme AIOps

Tableau 1.1 : Analyse des parties prenantes

Intérêt Influence Attente

Partie prenante

Élevé Élevée Réduire le temps que les ingénieurs consacrent au triage des échecs de pipeline

Responsable

unité DevOps

Élevé Moyenne Diagnostic plus rapide et plus clair sans devoir apprendre un nouvel outil

Ingénieurs

delivery client

Moyen Élevée Méthodologie rigoureuse, résultats mesurables, architecture solide

Encadrante

académique (ESPRIT)

Moyen Moyenne Pas de fuite de secrets vers les API LLM tierces

Responsable

sécurité/conformité

Élevé Moyenne Livrer un projet de fin d'études de niveau

production, défendable

Stagiaire PFE

(auteur)

Chapitre 1. Contexte général et problématique Plateforme AIOps

1.2.3 Analyse SWOT de l'initiative

Tableau 1.2 : Analyse SWOT

Forces Automatisation de bout en bout (ingestion jusqu'à la suggestion de remédiation) ; la base de connaissances auto-alimentée réduit le coût LLM récurrent dans le temps ; entièrement conteneurisé et reproductible sur Kubernetes

Faiblesses Dépendance à un fournisseur LLM tiers pour l'analyse centrale ; déploiement mono-réplica dans l'environnement de lab actuel ; couverture de tests automatisés limitée au-delà des tests unitaires

Opportunités Extensible à d'autres plateformes CI/CD (GitHub Actions, Azure DevOps) ; intégration potentielle aux stacks d'observabilité existantes (Prometheus/Grafana) déjà utilisées par la société

Menaces Évolution des tarifs ou des limites de débit de l'API LLM ; contraintes de confidentialité des données sur l'envoi d'extraits de logs à une API externe pour des projets clients sous clauses de conformité strictes

Ces faiblesses et menaces sont reprises en détail, avec des mitigations concrètes, dans le chapitre dédié à la Discussion critique à la fin de ce rapport.

1.3 Étude de l'existant

Dans la pratique actuelle, le diagnostic d'un pipeline en échec repose sur un processus entièrement manuel. L'ingénieur doit :

- parcourir manuellement des milliers de lignes de logs CI ;
- croiser les résultats de plusieurs outils indépendants (GitLab CI, SonarQube, Trivy) dans des interfaces séparées ;
- identifier la cause racine sans aucune assistance automatisée ;
- gérer les incidents sans traçabilité centralisée ;
- perdre du temps sur des problèmes récurrents dont les solutions ne sont pas documentées.

Par ailleurs, plusieurs outils existent sur le marché mais restent cloisonnés : GitLab CI

fournit des logs, SonarQube analyse la qualité du code, Trivy détecte les vulnérabilités d'images, tandis que les stacks d'observabilité classiques (Prometheus, Grafana, ELK) se focalisent sur les métriques d'infrastructure plutôt que sur le diagnostic de cause racine au niveau des pipelines. Les plateformes APM/AIOps commerciales (Datadog, Dynatrace) offrent des capacités avancées mais restent coûteuses, génériques et non adaptées à l'analyse spécifique des échecs CI/CD.

Chapitre 1. Contexte général et problématique Plateforme AIOps

1.4 Critique de l'existant

L'analyse met en évidence les limitations suivantes :

- Des données dispersées qui ne sont jamais corrélées entre les sources ;
- Aucune analyse de cause racine automatisée et contextualisée ;
- Aucune mémorisation des solutions aux problèmes récurrents ;
- Une alerting sans diagnostic, qui signale les échecs sans en expliquer l'origine ;
- Un temps de diagnostic élevé (30 à 90 minutes) et une faible traçabilité des incidents.

Pour rendre cette critique mesurable plutôt qu'anecdotique, un exercice de suivi du temps a été mené lors du premier sprint : cinq échecs de pipelines récents traités manuellement par l'équipe hôte ont été chronométrés de bout en bout et décomposés en phases dans le tableau 1.3. La décomposition montre que plus de la moitié du temps de diagnostic est consacré à la seule localisation des informations pertinentes entre les outils, avant même que commence le raisonnement sur la cause racine — précisément la phase qu'une plateforme AIOps peut compresser à quelques secondes.

Tableau 1.3 : Décomposition du temps de diagnostic manuel (moyenne de 5 incidents observés)

Temps moy. Observation

Phase

Localisation du job en échec 12–20 min Les logs sont paginés et non

et lecture des logs bruts 8–15 min recherchables entre les jobs

8–40 min

Vérification croisée des 2–15 min Connexion séparée, interface séparée, tableaux de bord SonarQube / aucun lien depuis le log CI

Trivy

Très variable ; répété pour des erreurs

Formulation et test déjà rencontrées

d'une hypothèse

Aucun enregistrement persistant et

Communication du résultat consultable pour la prochaine fois
(Slack, ticket)

Total 30–90 min Correspond à la plage indiquée dans
l'objectif O1

Chapitre 1. Contexte général et problématique Plateforme AIOps

1.5 Solution proposée

Pour remédier à ces limitations, nous proposons une plateforme AIOps qui automatise l'analyse des pipelines CI/CD, centralise les résultats de GitLab CI, SonarQube et Trivy dans une interface unique, mémorise les solutions dans une base de connaissances auto-alimentée, mesure en continu la santé des projets, et gère les incidents avec une traçabilité complète et des post-mortems générés par IA. Le tableau 1.4 résume les objectifs du projet.

Tableau 1.4 : Objectifs du projet

ID Description

- O1 Réduire le temps de diagnostic des pipelines de 30–90 min à moins de 2 min grâce à l'IA
- O2 Centraliser les données GitLab CI, SonarQube et Trivy dans une interface unique
- O3 Fournir une analyse de cause racine par IA avec suggestions de correction priorisées par sévérité
- O4 Détecter et alerter sur les vulnérabilités de sécurité des conteneurs (CVEs Trivy)
- O5 Apprendre les solutions récurrentes dans une base de connaissances, évitant les appels LLM répétés
- O6 Offrir un score de santé par projet pour un pilotage continu de la qualité
- O7 Gérer le cycle de vie des incidents avec post-mortems générés par IA
- O8 Évaluer le risque des Merge Requests avant fusion, avec un commentaire automatique
- O9 Produire des rapports hebdomadaires agrégés de métriques DevOps

1.6 Méthodologie de travail

Le projet a été conduit selon une approche agile inspirée de Scrum, organisée en huit sprints itératifs d'environ trois semaines chacun, répartis sur l'année académique 2025–2026. Chaque sprint livrait un incrément fonctionnel (ingestion webhook, analyse IA, tableau de bord, score de santé, incidents, base de connaissances, conteneurisation, CI/CD), validé avant de passer au suivant. La figure 1.2 présente le planning de projet résultant.

Oct Nov Déc Jan Fév Mar Avr Mai

S1 – Ingestion webhook & API core
S2 – Analyse IA (Groq/Claude)
S3 – Tableau de bord React
S4 – Score de santé & Incidents
S5 – Base de connaissances & Rapports
S6 – Tests automatisés (Vitest)
S7 – Docker, Helm, k3s
S8 – Pipeline CI/CD Jenkins & Rapport

Figure 1.2 : Diagramme de Gantt du projet (année académique 2025–2026)

Chapitre 1. Contexte général et problématique Plateforme AIOps

1.7 Conclusion

Ce chapitre a présenté le contexte du projet, l'organisme d'accueil et les limitations qui justifient la solution proposée. Avant de spécifier les besoins, le chapitre suivant passe en revue la manière dont les travaux commerciaux et académiques existants adressent — partiellement — le même problème, afin de positionner précisément la contribution de ce projet.

CHAPITRE 2

État de l'art et étude comparative

2.1 Introduction

L'AIOps est un domaine commercial et de recherche actif. Ce chapitre passe en revue les principales catégories de solutions existantes — suites AIOps/APM commerciales, outillage natif CI/CD, et travaux académiques sur l'analyse de logs par RCA — et positionne la contribution de ce projet par rapport à chacune.

2.2 Plateformes AIOps et d'observabilité commerciales

2.2.1 Datadog et Dynatrace

Datadog et Dynatrace sont les deux suites commerciales d'Application Performance Monitoring (APM) dominantes, dotées de modules AIOps (Datadog Watchdog, Dynatrace Davis AI). Toutes deux excellent dans la détection d'anomalies au niveau infrastructure et applicatif (pics de latence, déviations de taux d'erreur) à l'aide de modèles statistiques et ML entraînés sur des métriques et des traces. Cependant, aucune n'est conçue autour de la sémantique des pipelines CI/CD : elles n'analysent pas nativement les logs de jobs GitLab/GitHub CI, ne croisent pas une quality gate SonarQube avec un scan CVE Trivy pour le même commit, et leurs narratives de cause racine sont générées à partir d'une corrélation de télémétrie plutôt que d'une lecture IA générative du contexte d'échec réel. Elles sont également facturées par hôte/conteneur ingéré, ce qui les rend coûteuses pour un cas d'usage centré sur les pipelines CI plutôt que sur l'infrastructure de production.

2.2.2 Moogsoft et BigPanda

Moogsoft et BigPanda se focalisent sur la corrélation d'alertes : dédupliquer et regrouper des milliers d'alertes de supervision en un nombre plus réduit d'incidents actionnables. Ceci est conceptuellement proche du service de corrélation implémenté dans ce projet, mais opère à nouveau au niveau des alertes infrastructure, non au niveau des échecs de pipelines CI/CD, et ne produit ni explication de cause racine en langage naturel ni suggestion de correction au niveau du code.

Chapitre 2. État de l'art et étude comparative Plateforme AIOps

2.2.3 PagerDuty / Jeli et outils de gestion d'incidents

PagerDuty (et son acquisition Jeli) fournissent une gestion mature du cycle de vie des incidents avec des timelines, une escalade on-call et des revues post-incidents —

directement comparables au modèle d'Incident et à la fonctionnalité de post-mortem de ce projet. Leurs post-mortems, cependant, restent des templates rédigés manuellement ; ils ne génèrent pas automatiquement un brouillon de post-mortem depuis un LLM lisant la timeline de l'incident et la cause racine technique associée, ce que ce projet automatise.

2.3 Outillage natif CI/CD

GitLab CI, SonarQube et Trivy — les trois sources de données intégrées par cette plateforme — résolvent chacun une tranche du problème de façon isolée : GitLab CI exécute et journalise le pipeline, SonarQube évalue la qualité statique du code, et Trivy scanne les images de conteneurs à la recherche de CVEs connues. Aucun des trois ne corrèle sa propre sortie avec les deux autres, et aucun ne raisonne sur la cause racine d'un échec ; ils rapportent ce qui s'est passé, jamais pourquoi, ni ce qu'il convient de faire. Le propre « Copilot for pipelines » de GitHub / les résumés d'échec GitHub Actions (annoncés en 2024) pointent dans la même direction que ce projet, mais restent liés à l'écosystème GitHub et, au moment de la rédaction, ne résument qu'un seul log de job sans corrélation multi-outils ni base de connaissances persistante et consultable.

2.4 État de l'art académique : LLMs pour l'analyse de logs et la RCA

Des travaux académiques récents (notamment des études sur la détection d'anomalies de type LogGPT et sur l'utilisation des LLMs pour la résumé de cause racine d'incidents dans des opérations cloud à grande échelle [1]) confirment deux résultats directement pertinents pour la conception de ce projet : (1) les LLMs à usage général, lorsqu'on leur fournit un contexte bien structuré (extraits de logs pertinents plutôt que logs bruts complets), se comportent de manière compétitive par rapport aux modèles spécialisés d'analyse de logs en classification de cause racine — ce qui justifie l'étape de normalisation/corrélation avant l'appel LLM plutôt que d'alimenter directement les logs bruts ; et (2) la mise en cache/mémorisation des patterns déjà diagnostiqués réduit substantiellement à la fois le coût et la latence pour les échecs récurrents — la justification même du service de base de connaissances de ce projet.

Chapitre 2. État de l'art et étude comparative Plateforme AIOps

2.5 Synthèse comparative

Le tableau 2.1 résume la comparaison selon les dimensions les plus pertinentes pour les objectifs du projet (tableau 1.4).

Tableau 2.1 : Synthèse comparative des approches existantes

Natif	Corrélation	RCA	IA	Base de Profil	de coût
CI/CD	multi-source	(générative)	connaiss.		
Solution	auto-alimentée				
Datadog	/ Non	Partielle	Non	Non	Élevé (par hôte)
Dynatrace	(infra	(statistique)			
seulement)					
Moogsoft	/ Non	Oui	Non	Non	Élevé
BigPanda	(alertes)				
PagerDuty	/ Partiel	Non	Non	Non	Moyen
Jeli	(manuel)				
GitHub Copilot	Oui	Non	Partielle	Non	Inclus dans
for CI	(source le plan				
unique)					
Ce projet	Oui	Oui	Oui	Oui	Faible (API LLM
(GitLab+	(LLM) + cache)				
Sonar+Trivy)					

2.6 Positionnement de la contribution

Ce projet ne prétend pas surpasser les suites APM commerciales matures dans la détection d'anomalies au niveau infrastructure — ce n'est pas son objectif. Sa contribution est plus ciblée et spécifique : une plateforme de cause racine native CI/CD, corrélée multi-sources, assistée par IA générative, avec un mécanisme de mémoire organisationnelle (la base de connaissances), délivrée à une fraction du coût de licence des suites AIOps commerciales, car elle repose sur des API LLM à la consommation plutôt que sur des agents de supervision par hôte. Ce positionnement a directement motivé les choix architecturaux détaillés au chapitre 4.

Chapitre 2. État de l'art et étude comparative Plateforme AIOps

2.7 Conclusion

Cet état de l'art confirme qu'aucun outil existant ne couvre l'intersection spécifique de capacités ciblées par les objectifs O1–O9 : ingestion native CI/CD, corrélation multi-sources, analyse de cause racine générative, et base de connaissances auto-alimentée. Le chapitre suivant formalise les besoins qui opérationnalisent ce positionnement.

CHAPITRE 3

Analyse et spécification des besoins

3.1 Introduction

Ce chapitre identifie les acteurs de la plateforme, liste les besoins fonctionnels et non fonctionnels, et les formalise à travers une modélisation par cas d'utilisation.

3.2 Identification des acteurs

La plateforme s'appuie sur un Contrôle d'Accès Basé sur les Rôles (RBAC) avec trois niveaux :

- Administrateur : toutes les opérations (suppression, recalcul, configuration) ;
- Analyste : analyses, résolution, commentaires et post-mortems ;
- Observateur : accès en lecture seule.

Un acteur système intervient également : le serveur GitLab, qui émet des événements de pipeline et de merge request via des webhooks.

3.3 Besoins fonctionnels

3.3.1 Tableau de bord

Le tableau de bord est la page d'accueil du système et doit donner à un ingénieur, dans les secondes suivant sa connexion, une vue synthétique de l'état de tous les projets supervisés : la liste des pipelines récents avec leur statut, le taux de succès global sur les sept derniers jours sous forme de graphique de tendance, un ensemble d'indicateurs clés de performance (incidents actifs, MTTR moyen, vulnérabilités critiques ouvertes), une liste classée des problèmes récurrents les plus fréquents entre projets, et un flux live de notifications poussées via WebSocket au fur et à mesure que de nouvelles analyses se complètent. Le tableau de bord doit rester utilisable avec un filtre global par projet, car un ingénieur ne s'intéresse généralement qu'à un sous-ensemble des projets supervisés.

Chapitre 3. Analyse et spécification des besoins Plateforme AIOps

3.3.2 Gestion des pipelines

Le système doit exposer une liste paginée et filtrable de pipelines (filtrable par statut, branche, projet et plage de dates), car le nombre de pipelines croît en continu et une liste non bornée ne serait pas scalable. La sélection d'un pipeline doit révéler son détail complet : la liste des jobs et leur statut individuel, les logs bruts des jobs en échec, et — lorsque disponible — l'analyse IA associée. Un analyste doit également pouvoir

déclencher manuellement la ré-analyse d'un pipeline donné, par exemple après avoir corrigé la configuration d'un outil sous-jacent.

3.3.3 Analyse IA – Analyse de cause racine

Pour chaque pipeline analysé, le système doit classer l'échec dans l'un des types d'erreur d'un ensemble fixe (échec de build, échec de tests, problème de dépendance, vulnérabilité de sécurité, erreur de configuration, ou inconnu), produire une explication en langage naturel de la cause racine, assigner un niveau de risque (critique/élevé/moyen/faible) accompagné d'un score de confiance numérique, et proposer une ou plusieurs suggestions de correction priorisées, portant chacune optionnellement une commande shell prête à l'emploi ou un indice de code.

3.3.4 Sécurité et qualité du code

Le système doit intégrer les résultats du scan Trivy, exposant des compteurs par sévérité, la liste complète des CVEs détectées avec les informations de paquet et de version corrigée, et un workflow pour marquer une vulnérabilité comme acceptée ou ignorée. Il doit également intégrer les métriques SonarQube — statut de la quality gate, nombre de bugs, code smells et couverture de tests — car ces données alimentent directement le calcul du score de santé et le contexte de l'analyse IA.

3.3.5 Base de connaissances

Chaque fois qu'un analyste résout une analyse, le système doit automatiquement mémoriser la solution validée sous une signature unique dérivée du type d'erreur et de la cause racine, de sorte qu'une occurrence future soit répondue instantanément depuis le cache. La base de connaissances doit également être consultable et recherchable indépendamment (recherche plein texte sur titre, tags, type d'erreur et cause racine).

Chapitre 3. Analyse et spécification des besoins Plateforme AIOps

3.3.6 Score de santé, incidents et rapports

Le système doit calculer en continu, par projet, un score de santé composite entre 0 et 100, converti en grade de A à F, et doit exposer son détail complet (les six métriques pondérées du tableau 4.3). Les pipelines en échec doivent automatiquement ouvrir un incident, que le système suit à travers un cycle de vie (ouvert -> en investigation -> résolu) avec une timeline horodatée, un calcul du MTTR, et un post-mortem généré par IA à la demande. Les merge requests ouvertes doivent recevoir un commentaire automatique de risque. Enfin, le système doit produire un rapport hebdomadaire agrégé par projet, comparable semaine après semaine.

3.4 Besoins non fonctionnels

- Sécurité : authentification JWT, RBAC à trois niveaux, validation HMAC-SHA256, hachage Bcrypt, en-têtes HTTP de sécurité.
- Performance : analyse de pipeline en moins de dix secondes.
- Temps réel : mise à jour du tableau de bord via WebSocket.
- Scalabilité : traitement asynchrone via files de messages et workers concurrents.
- Fiabilité : reprise sur erreur avec relances et backoff exponentiel.
- Maintenabilité : architecture en couches, code modulaire.
- Portabilité : conteneurisation complète et déploiement Kubernetes reproductible.

Tableau 3.1 : Besoins non fonctionnels avec critères d'acceptation mesurables

BNF Critère d'acceptation mesurable Vérification

Sécurité 100% des webhooks avec HMAC invalide rejetés Test de pénétration avec HTTP 401 ; mots de passe jamais en clair + revue de code

Performance Analyse de bout en bout < 10 s sur 95% Journalisation des pipelines observés sur 20+ runs réels

Temps réel Tableau de bord reflète nouvelle analyse Mesure onglet dans les 500 ms réseau navigateur

Scalabilité Backend sans état ; 2 réplicas du worker Test de charge sans duplication de jobs kubectrl scale

Fiabilité Échec transitoire API retente jusqu'à 3 fois Injection de pannes

Maintenabilité Chaque service <= 10 fonctions publiques, Revue de code une responsabilité unique

Portabilité Stack démarre depuis k3s vierge avec Re-déploiement une seule commande helm install depuis zéro

3.5 Comportement fonctionnel : diagramme d'activité

La figure 3.1 modélise la logique de décision du worker d'analyse de pipeline sous forme de diagramme d'activité UML, rendant explicites les deux branches conditionnelles : la branche cache hit/miss de la base de connaissances, et la branche succès/échec du pipeline qui détermine si un incident est créé.

3.6 Modélisation des cas d'utilisation

La figure 3.2 présente le diagramme de cas d'utilisation global. Les flèches de généralisation expriment qu'Analyste et Observateur héritent des capacités en lecture seule d'un Utilisateur Authentifié de base, tandis qu'Administrateur hérite en plus de

toutes les capacités de l'Analyste.

3.6.1 Raffinement des cas d'utilisation

Tableau 3.2 : « Analyser un pipeline en échec »

Elément Description

Acteur principal Serveur GitLab (système), Analyste

Acteurs sec. LLM Groq/Claude, SonarQube, Trivy

Précondition Pipeline terminé sur projet supervisé ; secret webhook configuré

Scénario 1. GitLab émet webhook signé. 2. Validation signature HMAC.

3. Job mis en file (202 Accepted). 4. Worker collecte logs, issues SonarQube, CVEs Trivy et commits. 5. Normalisation et corrélation. 6. «include» KB interrogée par signature.

7. Cache miss : LLM classe l'erreur, produit cause racine et suggestions. 8. Résultat persisté et diffusé en temps réel.

Postcondition Analyse disponible ; incident auto-crée si pipeline en échec

Flux alt. Cache hit : solution mémorisée réutilisée, LLM non appelé

Exceptions HMAC invalide (401) ; timeout API (retente 3 fois, backoff exp.)

Tableau 3.3 : « Résoudre une analyse IA »

Elément Description

Acteur principal Analyste (rôle >= analyst)

Précondition Analyse non résolue existe

Scénario 1. Ouverture page détail pipeline. 2. Examen cause racine IA et application du correctif. 3. PATCH /api/analyses/:id/resolve.

4. «extend» KB alimentée par la solution validée.

5. MTTR calculé et stocké.

Postcondition resolved=true ; entrée KB créée ou mise à jour

Exceptions 403 Forbidden (rôle insuffisant) ; déjà résolu (no-op)

Tableau 3.4 : « Gérer un incident et générer un post-mortem »

Elément Description

Acteur principal Analyste

Précondition Incident auto-crée depuis pipeline en échec

Scénario 1. Ouverture détail incident (timeline, analyse liée).

2. Mise à jour statut (ouvert -> investigation -> résolu), chaque changement ajouté à la timeline. 3. Commentaires libres possibles. 4. Déclenchement génération post-mortem.

5. «extend» LLM résume timeline + analyse en post-mortem

structuré avec recommandations.

Postcondition Timeline complète ; post-mortem attaché à l'incident

Exceptions Post-mortem demandé avant résolution (rejeté) ; échec LLM

(relance, puis dégradé vers template)

Tableau 3.5 : « Commenter une MR avec évaluation de risque »

Elément Description

Acteur principal Serveur GitLab (système)

Acteur sec. LLM Groq/Claude

Précondition Webhook MR (action open ou update) reçu

Scénario 1. Identification dernière analyse pour le projet cible.

2. Calcul score de risque 0-100 (sévérité + confiance).

3. «include» authentification API GitLab via token stocké.

4. LLM génère explication en 3 phrases.

5. Commentaire Markdown posté sur la MR.

Postcondition MR affiche commentaire automatique de risque

Exceptions Pas encore d'analyse (commentaire ignoré) ; rate limit GitLab

(différé et relancé)

3.7 Matrice de traçabilité des besoins

Tableau 3.6 : Matrice objectifs -> fonctionnalités -> cas d'utilisation

Obj. Fonctionnalité(s) implémentée(s) Cas d'utilisation

O1 Analyseur cause racine IA, pipeline asynchrone BullMQ Tableau 3.2

O2 Services GitLab/SonarQube/Trivy + normaliseur Tableau 3.2

O3 ai/rootCause.analyzer.js, fixSuggester.js Tableau 3.3

O4 Service Trivy, modèle Vulnerability, panneau sécurité Tableau 3.2

O5 Service KB, cache par signature Figure 4.6

O6 Service score de santé (6 métriques pondérées, A-F) Section 4.6.1

O7 Modèle Incident, timeline, post-mortem IA Tableau 3.4

O8 Service MR-comment, scoring de risque Tableau 3.5

O9 Service rapport hebdomadaire, comparaison S/S Section 4.6.3

3.8 Conclusion

Ce chapitre a formalisé les besoins de la plateforme à travers l'identification des acteurs, la spécification fonctionnelle et non fonctionnelle, la modélisation par cas d'utilisation et une matrice de traçabilité. Le chapitre suivant traduit ces besoins en une conception

technique détaillée.

CHAPITRE 4

Conception et architecture

4.1 Introduction

Ce chapitre présente l'architecture globale, le flux de traitement, les modèles de données, les services métier, et la conception de l'API et du frontend.

4.2 Architecture globale

La plateforme suit une architecture événementielle. GitLab émet un webhook à la fin de chaque pipeline (ou sur un événement de merge request) ; le backend valide la requête, la met en file dans une queue de messages, puis des workers asynchrones effectuent la collecte, la normalisation, la corrélation et l'analyse IA. Les résultats sont persistés dans MongoDB et poussés vers le tableau de bord React en temps réel via WebSocket. La figure 4.1 donne la vue globale de l'architecture.

GitLab CE SonarQube Trivy

Pipelines / MR Qualité Scanner CVE

webhook (HMAC)

Backend – Node.js / Express

Webhooks, API REST, workers BullMQ

Groq / Claude Redis MongoDB

LLM Cause Racine BullMQ Socket.io 7 collections

Frontend – React / Vite

Tableau de bord, Score de santé, Incidents, KB

Figure 4.1 : Architecture globale de la plateforme AIOps

4.2.1 Diagramme de composants

La figure 4.2 affine l'architecture en composants UML avec interfaces fournies/requises explicites, montrant comment le backend expose une API REST et un endpoint webhook tout en requérant les trois sources de données CI/CD externes et le fournisseur LLM.

4.2.2 Diagramme de déploiement physique

La figure 4.3 montre les nœuds de déploiement physique : le poste développeur (Windows 11), la VM Vagrant/VirtualBox (Ubuntu 22.04) hébergeant GitLab CE, Jenkins et le cluster k3s, et les dépendances SaaS externes (DockerHub, Groq, Claude).

4.3 Architecture backend

Le backend est une application Node.js (ESM) / Express organisée en couches :

1 backend/src/

2 |-- server.js # Point d'entrée HTTP (port 3001)

3 |-- app.js # App Express + middlewares globaux

4 |-- socket.js # Initialisation Socket.io

5 |-- config/ # env, base de données (Mongoose), redis

6 |-- api/ # contrôleurs, middlewares, routes

7 |-- ai/ # clients groq/claude, analyseur, prompts

8 |-- models/ # 7 schémas Mongoose

9 |-- queues/ # files et workers BullMQ

10 |-- services/ # gitlab, sonarqube, trivy, corrélation, ...

11 \-- utils/ # logger (Winston), httpClient, retry

Les couches sont : API (contrôleurs, middlewares, routes), services métier (GitLab, SonarQube, Trivy, normalisation, corrélation, score de santé, commentaire MR, rapport hebdomadaire), files et workers (BullMQ sur Redis), couche IA (clients LLM, orchestrateur de cause racine, prompts) et modèles (schémas Mongoose).

4.4 Flux de traitement principal

Lorsqu'un pipeline se termine, GitLab envoie un webhook à POST /api/webhooks/gitlab.

Le middleware HMAC valide la signature, un rate limiter plafonne les requêtes, et le contrôleur met en file un job dans la queue BullMQ d'analyse de pipeline, retournant immédiatement 202 Accepted. Un worker (5 concurrents) exécute ensuite les étapes du tableau 4.1.

Tableau 4.1 : Étapes de traitement de l'analyse de pipeline

Étape Action

1 Récupérer les logs GitLab CI des jobs en échec

2 Récupérer les issues SonarQube (bugs, vulnérabilités, quality gate)

3 Récupérer les CVEs Trivy depuis l'artefact trivy-results.json

4 Récupérer les derniers commits (auteur, message, diff)

5 Normaliser toutes les sources en un objet unifié

6 Recherche KB : cache hit réutilise la solution ; cache miss appelle le LLM

7 Persister l'analyse, upsert KB, recalculer le score de santé

7b Auto-crée un incident si le pipeline a échoué (un par pipeline)

7c Poster un commentaire de risque sur la MR GitLab si ouverte

8 Émettre un événement WebSocket pour mise à jour instantanée

4.5 Modèles de données

La base MongoDB s'appuie sur sept collections principales (tableau 4.2) :

Tableau 4.2 : Collections principales

Collection Rôle

User Comptes utilisateurs et rôles (RBAC)

Pipeline Métadonnées et statut des pipelines GitLab

Analysis Résultats des analyses IA (cause racine, suggestions, risque)

Vulnerability Vulnérabilités détectées par Trivy (CVEs)

KnowledgeBase Solutions mémorisées et signatures d'erreur

ProjectHealth Score de santé et détail par projet

Incident Incidents, timeline et post-mortems IA

Trois décisions de modélisation sont à noter. Premièrement, SuggestedFix, TimelineEntry, PostMortemData et ScoreItem sont modélisés comme sous-documents embarqués : ils n'ont pas d'identité propre hors de leur parent (composition UML), et MongoDB les récupère atomiquement en une seule requête. Deuxièmement, la relation Incident-Analysis est une référence (ObjectId) car un document Analysis est interrogé indépendamment du workflow incident. Troisièmement, ProjectHealth est calculé à partir de Pipeline, Analysis et Vulnerability sur un calendrier, acceptable car les scores de santé sont des indicateurs à cohérence éventuelle.

Le schéma Analysis (extrait) :

```

1 const analysisSchema = new Schema({
2   pipelineId: { type: String, required: true, index: true },
3   projectId: { type: String, required: true },
4   errorType: { type: String, enum: ['build_failure','test_failure',
5     'dependency_issue','security_vulnerability',
6     'configuration_error','unknown'] },
7   rootCause: String,
8   confidence: { type: Number, min: 0, max: 1 },
9   riskLevel: { type: String, enum: ['critical','high','medium','low'] },
10  suggestedFixes: [suggestedFixSchema],
11  affectedFiles: [String],
12  processingTime: Number,
13  resolved: { type: Boolean, default: false },
14  mttr: Number
15 }, { timestamps: true });

```

4.5.1 Diagramme de classes de la couche service

La logique métier est implémentée par un ensemble de services sans état (figure 4.6), chacun exposant un petit ensemble d'opérations publiques et dépendant de modèles Mongoose via des dépendances «use» UML.

4.6 Services métier

4.6.1 Service de score de santé

Chaque projet reçoit un score de 0 à 100 calculé à partir de six métriques pondérées, puis converti en grade de A à F.

Tableau 4.3 : Métriques et pondérations du score de santé

Métrique Poids Logique

Taux de succès pipelines 25% % de pipelines "success" sur 7 jours

Vulnérabilités critiques 20% 100 – (critique x 20 + haut x 10)

MTTR moyen 20% <15 min->100, <30->80, <60->60, sinon 30

Ancienneté du dernier échec 10% 7j->100, 3j->80, 1j->50, sinon 20

Couverture de tests 15% Basé sur données SonarQube

Code smells 10% 0->100, <5->80, <20->60, sinon 30

Tableau 4.4 : Correspondance score-grade

>=90 >=75 >=60 >=45 <45

A B C D F

4.6.2 Service de base de connaissances

Ce service mémorise les solutions aux erreurs récurrentes. Chaque entrée est indexée par une signature déterministe MD5(errorType + ":" + rootCause[:100]), normalisée en minuscules. Quand un analyste marque une analyse comme résolue, la solution est mémorisée ; lors d'une analyse similaire ultérieure, un cache hit injecte la solution connue en tête des suggestions, évitant un appel LLM redondant.

4.6.3 Autres services

Le service MR-comment calcule un score de risque 0-100 et poste une explication en français sur la merge request GitLab. Le service de rapport hebdomadaire agrège les volumes de pipelines, taux de succès, nouvelles vulnérabilités, MTTR moyen et comptages d'incidents avec comparaison semaine sur semaine. Le service de corrélation relie les paquets vulnérables mentionnés dans les logs aux issues SonarQube et aux fichiers CI en échec. Trois files BullMQ gèrent le travail asynchrone : pipeline-analysis (5 workers, 3 relances), log-collection (3 workers) et notifications (10 workers).

4.7 Conception de l'API REST

L'API expose des endpoints orientés ressources protégés par JWT et RBAC.

Tableau 4.5 : Principaux endpoints de l'API REST

Méthode Endpoint Rôle Description

POST /api/auth/login -- Login (accès 15 min + refresh 7 j)

POST /api/webhooks/gitlab HMAC Événements pipeline & MR GitLab

GET /api/pipelines viewer Liste paginée + filtres

GET /api/pipelines/stats viewer KPIs tableau de bord (7 jours)

POST /api/pipelines/:id/retry analyst Re-déclencher l'analyse

GET /api/analyses/recurring viewer Top 5 problèmes récurrents
PATCH /api/analyses/:id/resolve analyst Marquer résolu (alimente KB)
GET /api/vulnerabilities viewer Liste CVE (filtres)
GET /api/knowledge/search viewer Recherche plein texte KB
GET /api/health-score viewer Scores de tous les projets
GET /api/incidents/:id viewer Détail incident + timeline
POST /api/incidents/:id/postmortem analyst Générer post-mortem IA
GET /api/reports/current viewer Rapport de la semaine courante

4.7.1 Justification des choix de l'API

Trois décisions structurent chaque endpoint. Premièrement, la pagination est obligatoire sur chaque liste (?page=, ?limit=, plafonné à 100) pour éviter les scans MongoDB non bornés. Deuxièmement, les erreurs suivent une enveloppe unique { error: { code, message } } avec un code lisible par machine, pour que le frontend puisse brancher sur le code. Troisièmement, le webhook retourne 202 Accepted pour découpler la livraison GitLab (timeout 10 s) de la latence LLM.

4.7.2 Justification des choix de base de données

MongoDB a été choisi pour trois raisons. (1) La forme du document Analysis varie avec le type d'erreur ; un schéma flexible évite une table large avec des colonnes majoritairement nulles. (2) Les sous-documents embarqués sont toujours lus avec leur parent — le cas textbook pour l'embarquement. (3) Le pattern d'écriture est à prédominance append et ordonné dans le temps, ce qui joue en faveur de MongoDB. L'indexation suit les patterns de requête effectivement émis par le tableau de bord.

4.8 Vues comportementales complémentaires

Deux séquences complémentaires sont documentées. La figure 4.7 montre le flux de commentaire automatique sur une MR : réception webhook -> calcul score de risque -> génération explication LLM (3 phrases en français) -> post du commentaire Markdown. La figure 4.8 montre la résolution d'incident et génération de post-mortem : PATCH status=resolved -> ajout timeline -> POST /postmortem -> LLM résume timeline + analyse -> sauvegarde postMortem.

4.9 Architecture frontend

Le frontend est une application React (Vite) SPA. L'état global est partagé entre Redux Toolkit (tokens d'authentification, filtres actifs), TanStack Query (cache d'état serveur, requêtes et mutations) et un client Socket.io pour les événements temps réel.

Les événements reçus (analysis:complete, pipeline:new, incident:created) invalident les requêtes concernées, rafraîchissant le tableau de bord instantanément. L'application comprend onze pages et des familles de composants réutilisables.

4.10 Conclusion

Ce chapitre a détaillé la conception de la solution. Le chapitre suivant en présente l'implémentation, les tests et le déploiement.

CHAPITRE 5

Implémentation, tests et déploiement

5.1 Introduction

Ce chapitre décrit l'environnement de travail, la stack technologique, les principales interfaces, des extraits de code représentatifs, les tests automatisés, la conteneurisation et le déploiement Kubernetes, le pipeline CI/CD Jenkins, les mesures de sécurité et les résultats obtenus.

5.2 Environnement de travail

5.2.1 Environnement matériel

Le développement a été réalisé sur un poste de travail sous Windows 11 (build 26200), équipé d'un processeur Intel Core i5-11400H (6 cœurs / 12 threads, 2,70 GHz) et de 16 Go de RAM. Tous les services d'infrastructure nécessitant un hôte Linux — GitLab CE, GitLab Runner, Jenkins, le cluster Kubernetes k3s et SonarQube — s'exécutent dans une machine virtuelle Vagrant/VirtualBox dédiée (Ubuntu 22.04 LTS, 8 Go de RAM alloués, 4 vCPUs), isolant l'environnement de lab du poste hôte.

5.2.2 Environnement logiciel

Les principaux outils sont : Visual Studio Code comme IDE principal, Git 2.47 et GitHub pour le contrôle de versions, Node.js 20 LTS (runtime pour le backend et l'outillage frontend) et npm comme gestionnaire de paquets, Docker (Engine 29.x) et le plugin Docker Compose pour la conteneurisation, Vagrant et VirtualBox pour provisionner la VM de lab, GitLab CE et GitLab Runner pour l'hébergement des sources et les déclencheurs de pipelines, Jenkins LTS pour l'orchestration CI/CD, SonarQube Community Edition pour l'analyse statique, Trivy pour le scan de vulnérabilités des conteneurs, Helm 3 et kubectl pour les opérations Kubernetes, et ngrok pour exposer le backend local aux webhooks GitLab pendant le développement.

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.3 Stack technologique

Les tableaux 5.1, 5.2 et 5.3 récapitulent la stack.

Tableau 5.1 : Stack backend

Technologie Rôle

Node.js 20 / Express Runtime ESM et framework API REST

MongoDB / Mongoose Base de données document et ODM
 Redis / BullMQ Broker, rate limiting et jobs asynchrones
 Socket.io Communication WebSocket en temps réel
 Groq / Anthropic LLM principal (Llama 3.3 70B) et de secours (Claude)
 JWT / Bcrypt Authentification et hachage des mots de passe
 Helmet / Winston Sécurité HTTP et journalisation structurée
 Vitest Framework de tests unitaires

Tableau 5.2 : Stack frontend

Technologie Rôle
 React 18 / Vite Framework UI et outil de build
 React Router Routage SPA
 TanStack Query Cache d'état serveur et synchronisation
 Redux Toolkit État global (auth, filtres)
 Tailwind CSS Styles utilitaires
 Recharts Graphiques (ligne, barres, radar)
 Socket.io Client Mises à jour en temps réel

Tableau 5.3 : Stack infrastructure et DevOps

Composant Rôle
 Docker / DockerHub Conteneurisation et registre public d'images
 k3s (Kubernetes) Orchestrateur de conteneurs léger (VM Vagrant)
 Helm 3 Gestionnaire de paquets Kubernetes
 Jenkins LTS Serveur CI/CD avec pipeline déclaratif
 GitLab CE / Runner Serveur Git local et CI/CD
 SonarQube CE / Trivy Analyse statique et scan CVE
 ngrok Tunnel HTTPS pour les webhooks GitLab locaux

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.4 Interfaces de l'application

[Emplacement figure – insérer capture du tableau de bord principal]

Interface – Tableau de bord principal

Figure 5.1 : Interface – Tableau de bord principal

[Emplacement figure – insérer capture de l'analyse IA]

Interface – Détail d'une analyse IA

Figure 5.2 : Interface – Détail d'une analyse IA

[Emplacement figure – insérer capture du panneau sécurité]

Interface – Panneau sécurité (vulnérabilités)

Figure 5.3 : Interface – Panneau sécurité (vulnérabilités)

Remplacer ces trois emplacements par des captures d'écran réelles de l'application en fonctionnement : (1) la page /dashboard montrant les KPIs et le graphique de tendance sur 7 jours, (2) la page /pipelines/:id montrant une carte d'analyse IA avec cause racine et suggestions, (3) la page /security montrant le tableau des vulnérabilités Trivy.

5.5 Extraits de code clés

5.5.1 Validation de la signature webhook

La réception des webhooks est sécurisée par une signature HMAC-SHA256, comparée en temps constant pour prévenir les attaques par timing.

```

1 const crypto = require("crypto");

3 function verifyGitlabSignature(req, res, next) {

4   const received = Buffer.from(req.headers["x-gitlab-token"] || "");

5   const expected = Buffer.from(crypto

6     .createHmac("sha256", process.env.GITLAB_WEBHOOK_SECRET)

7     .update(req.rawBody)

8     .digest("hex"));

9   if (received.length !== expected.length ||

10     !crypto.timingSafeEqual(received, expected)) {

11     return res.status(401).json({ error: "Invalid signature" });

12   }

13   next();

14 }
```

Listing 5.1 : Middleware de validation HMAC (extrait)

5.5.2 Calcul pondéré du score de santé

```
1 function computeHealthScore(m) {  
2   const score =  
3     0.25 * m.pipelineSuccessRate  
4   + 0.20 * m.criticalVulnsScore  
5   + 0.20 * m.mttrScore  
6   + 0.15 * m.coverageScore  
7   + 0.10 * m.lastFailureScore  
8   + 0.10 * m.codeSmellsScore;  
9   return scoreToGrade(Math.round(score)); // 0..100 -> A..F  
10 }
```

Listing 5.2 : Calcul pondéré du score de santé (extrait)

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.5.3 Worker BullMQ avec concurrence bornée et relances

Le worker d'analyse est enregistré une seule fois au démarrage avec une concurrence fixe et une politique de relance avec backoff exponentiel, de sorte qu'un échec transitoire d'une API externe (GitLab, SonarQube) ne fasse pas échouer définitivement le job.

```
1 const worker = new Worker('pipeline-analysis', async (job) => {  
2   const { pipelineId, projectId } = job.data;  
3   const logs = await gitlabService.getFailedJobLogs(projectId,  
4     pipelineId);  
5   const sonar = await sonarqubeService.getIssues(projectId);  
6   const trivy = await trivyService.getReport(projectId, pipelineId);  
7   const normalized = normalizerService.normalize({ logs, sonar, trivy  
8     });
```

```

7 const cached = await
knowledgeBaseService.findCachedSolution(normalized);

8 const result = cached ?? await
rootCauseAnalyzer.analyze(normalized);

9 await Analysis.create({ pipelineId, projectId, ...result });

10 await healthScoreService.computeScore(projectId);

11 io.emit('analysis:complete', { pipelineId, projectId });
12 }, {

13 connection: redisConnection,

14 concurrency: 5,
15 });
16 worker.on('failed', (job, err) => logger.error(`Job ${job.id} failed`, err));

```

Listing 5.3 : Enregistrement du worker d'analyse (extrait)

5.5.4 Connexion MongoDB résiliente

Le backend renouvelle sa tentative de connexion MongoDB avec un backoff exponentiel borné plutôt que de planter, car dans un déploiement Kubernetes le pod mongodb peut ne pas être prêt quand le pod aiops-backend démarre.

```

1 async function connectWithRetry(uri, attempt = 1) {

2 try {

3 await mongoose.connect(uri, { serverSelectionTimeoutMS: 5000 });

4 logger.info('MongoDB connecté');

5 } catch (err) {

6 const delay = Math.min(1000 * 2 ** attempt, 30000);

7 logger.warn(`Connexion MongoDB échouée (tentative ${attempt}),

8 nouvelle tentative dans ${delay}ms`);

```

```

9 setTimeout(() => connectWithRetry(uri, attempt + 1), delay);
10 }
11 }

```

Listing 5.4 : Connexion Mongoose avec relance (extrait)

5.5.5 Récupération de données en temps réel côté frontend

TanStack Query combine polling et invalidation de cache déclenchée par les événements

Socket.io, pour que le tableau de bord reste à jour sans polling naïf à intervalle fixe

une fois la connexion WebSocket établie.

```

1 export function usePipelines(filters) {
2   const queryClient = useQueryClient();
3   useEffect(() => {
4     const socket = getSocket();
5     const onNewPipeline = () =>
      queryClient.invalidateQueries(['pipelines']);
6     socket.on('pipeline:new', onNewPipeline);
7     return () => socket.off('pipeline:new', onNewPipeline);
8   }, [queryClient]);
9   return useQuery({
10    queryKey: ['pipelines', filters],
11    queryFn: () => pipelinesApi.getAll(filters),
12    staleTime: 30_000,
13  });
14 }

```

Listing 5.5 : Hook usePipelines avec invalidation par Socket.io (extrait)

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.6 Tests automatisés

Les tests unitaires utilisent Vitest (ESM natif, intégré à Vite). La suite comprend 25 tests résumés dans le tableau 5.4.

Tableau 5.4 : Suite de tests automatisés

Fichier de test Nb Périmètre

healthScore.service.test.js 12 Fonctions de scoring, correspondance grade, cas limites

knowledgeBase.service.test.js 6 Déterminisme signature, extraction de tags

Incidents.test.jsx 3 Formatage timeSince, agrégation buildHistory

HealthScore.test.jsx 2 Rendu de composant React

Spinner.test.jsx 2 Rendu de composant React

Total 25 100% passant

La suite backend (18 tests) s'exécute en moins de sept secondes ; la suite frontend (7 tests) utilise @testing-library/react et jsdom. Les deux suites s'exécutent automatiquement dans le pipeline Jenkins.

5.6.1 Stratégie de test et pyramide des tests

Les 25 tests automatisés ciblent la couche basse et la plus rapide de la pyramide de tests classique : fonctions pures et composants isolés sans dépendance réseau ou base de données (figure 5.4). C'était une décision de périmètre délibérée sous les contraintes de temps d'un PFE en solo : les tests unitaires sur la logique de scoring et la génération de signature capturent les régressions les plus risquées (un poids de score de santé incorrect ou une signature de base de connaissances non déterministe corrompraient silencieusement les données pour tout projet) à coût quasi nul, alors que les tests d'intégration sur MongoDB/Redis réels et les tests end-to-end en navigateur auraient nécessité un environnement de test dédié et plus de temps CI que disponible.

E2E – planifiés

Tests d'intégration – planifiés

Tests unitaires – 25 (implémentés)

Figure 5.4 : Pyramide des tests : couverture implémentée vs extensions planifiées

5.6.2 Ce qui n'est intentionnellement pas couvert

Tableau 5.5 : Lacunes de couverture et mitigation manuelle

Non automatisé Mitigation manuelle / risque résiduel

Validation HMAC sous vraies Testé manuellement avec de vrais webhooks GitLab

charges GitLab sur 5 sprints ; risque : drift de format de payload

non détecté automatiquement

Concurrence et relances Échecs déclenchés manuellement via timeouts API

du worker BullMQ forcés ; risque : conditions de course sous haute

concurrence non vérifiées

Intégration composants React Chaque page exercée manuellement dans le navigateur

(niveau page) à chaque sprint ; risque : régressions visuelles non

détectées automatiquement

Rendu chart Helm et rollout Vérifié via helm template + kubectl rollout status

Kubernetes manuel à chaque déploiement ; risque : une faute de

frappe dans values.yaml peut passer en revue

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.7 Conteneurisation

Chaque service est livré sous forme d'image Docker. Le frontend utilise un build

multi-étapes qui compile la SPA et la sert avec nginx, qui reverse-proxifie également

le trafic API et WebSocket vers le service backend.

1 FROM node:20-alpine AS builder

2 WORKDIR /app

3 COPY package*.json ./

4 RUN npm ci

5 COPY . .

6 RUN npm run build # -> dist/ (SPA statique)

8 FROM nginx:alpine

9 COPY --from=builder /app/dist /usr/share/nginx/html

10 COPY nginx.conf /etc/nginx/conf.d/default.conf

11 EXPOSE 80

12 CMD ["nginx", "-g", "daemon off;"]

Listing 5.6 : Dockerfile multi-étapes du frontend

5.8 Déploiement Kubernetes avec Helm

La plateforme est déployée sur un cluster Kubernetes léger (k3s) via un chart Helm. Le chart définit des déploiements pour le backend, le frontend (nginx) et MongoDB, un StatefulSet Redis, des services ClusterIP, un stockage persistant pour MongoDB, et un ingress Traefik. Les valeurs sensibles (secret JWT, token GitLab, clés API) sont conservées dans un `values.secrets.yaml` exclu du contrôle de version et injectées comme secrets Kubernetes. La figure 5.5 montre la topologie résultante du cluster.

Cluster k3s (namespace : default)

Traefik Ingress

aiops.local

Deployment Deployment

aiops-frontend (nginx) aiops-backend (Node.js)

Service ClusterIP proxy /api Service ClusterIP

:80 :3001

Deployment StatefulSet

mongodb + PVC 2Gi redis-master

Figure 5.5 : Architecture de déploiement Kubernetes (k3s)

Extrait du fichier `values.yaml` (Listing 5.7) :

1 backend:

2 image: { repository: khalil512/aiops-backend, tag: latest }

3 replicas: 1

4 port: 3001

5 resources:

6 requests: { cpu: 100m, memory: 256Mi }

7 limits: { cpu: 500m, memory: 512Mi }

8 config:

9 nodeEnv: production

10 mongodbUri: mongodb://mongodb:27017/aiops

11 redisHost: redis-master

12 groqModel: llama-3.3-70b-versatile

13 ingress:

14 enabled: true

15 className: traefik

16 host: aiops.local

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.9 Pipeline CI/CD Jenkins

Un pipeline Jenkins déclaratif automatise la chaîne de livraison complète. Jenkins s'exécute dans un conteneur avec accès au daemon Docker, à kubectl et helm, et au kubeconfig k3s. Le listing 5.8 montre les étapes du pipeline (condensé) et la figure 5.6 le flux de bout en bout correspondant.

1 pipeline {

2 agent any

**3 environment { DOCKERHUB_TOKEN =
credentials('dockerhub-token') }**

4 stages {

5 stage('Installer & tester backend') {

6 steps { sh 'cd backend && npm ci && npm test' }

7 }

8 stage('Installer & tester frontend') {

9 steps { sh 'cd frontend && npm ci && npm test' }

10 }


```
11 stage('Build images') {
12 steps {
13 sh 'docker build -t khalil512/aiops-backend:latest backend'
14 sh 'docker build -t khalil512/aiops-frontend:latest frontend'
15 }
16 }
17 stage('Pousser images') {
18 steps { sh 'echo $DOCKERHUB_TOKEN | docker login -u khalil512
19 --password-stdin && docker push khalil512/aiops-backend:latest
20 && docker push khalil512/aiops-frontend:latest' }
21 }
22 stage('Déployer sur k3s') {
23 steps { sh 'helm upgrade aiops ./helm/aiops -f values.yaml -f
24 $SECRETS_FILE --wait --timeout 5m && kubectl rollout
25 restart deployment/aiops-backend deployment/aiops-frontend' }
26 }
27 }
28 }
```

Listing 5.8 : Étapes du Jenkinsfile (condensé)

Le pipeline récupère les sources depuis GitHub, installe les dépendances, exécute les 25 tests, construit et pousse les deux images Docker vers DockerHub, puis met à jour la release Helm et redémarre les déploiements sur k3s. Une exécution complète prend environ quinze minutes initialement et dix minutes lors des exécutions suivantes grâce au cache des couches Docker.

GitHub Installer & Build Docker Poussier helm upgrade
 (checkout) tester (Vitest) images DockerHub + rollout restart
 Cluster k3s
 (pods mis à jour)

Figure 5.6 : Flux CI/CD de bout en bout

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.10 Défis d'ingénierie et solutions

La mise en place d'un environnement de lab auto-hébergé GitLab/Jenkins/k3s a mis en évidence plusieurs problèmes d'infrastructure non évidents dont la résolution est documentée ici, à la fois comme preuve de l'effort de débogage impliqué et comme référence pour reproduire l'environnement. Le tableau 5.6 résume les plus instructifs.

Tableau 5.6 : Problèmes d'infrastructure sélectionnés et leurs résolutions

Symptôme Cause racine et correction

Conteneur Le kubeconfig pointait vers 127.0.0.1, qui dans un réseau Docker Jenkins ne bridgé résout vers le loopback propre du conteneur, pas l'hôte VM. pouvait pas Corrigé en créant le conteneur Jenkins avec --network host afin d'atteindre le qu'il partage l'espace réseau de la VM.

serveur API k3s

(connexion

refusée sur

127.0.0.1:6443)

npm: not found L'image officielle jenkins/jenkins:its n'embarque aucun runtime quand le Node.js. Corrigé en installant Node.js 20 directement dans le pipeline Jenkins conteneur en cours d'exécution via le script NodeSource ; documenté lançait npm test comme non persistant entre les re-créations du conteneur.

Helm upgrade Avec le tag :latest, la spec Deployment est byte-identique après réussi mais pods helm upgrade, donc Kubernetes ne détecte aucun changement et continuaient à ignorer le rollout. Corrigé en enchaînant kubectl rollout restart servir l'ancien immédiatement après chaque helm upgrade dans le Jenkinsfile.
code

Appels API Git Bash et Node.js résolvent /tmp vers différents chemins physiques Jenkins REST sous Windows, donc le CSRF crumb récupéré par un outil était retournaient 403 invisible à l'autre. Corrigé en stockant le crumb et le cookie jar

No valid crumb dans un chemin sur le système de fichiers Windows partagé, accessible identiquement par les deux outils.

Kubernetes Le proxy était lié à 127.0.0.1 dans la VM ; le NAT de VirtualBox Dashboard délivre le trafic via l'interface réseau virtuelle du guest, qui n'atteint injoignable via jamais un service lié au loopback uniquement. Corrigé en reliant le port forward le proxy à 0.0.0.0 comme service systemd persistant, survivant aux VirtualBox redémarrages de la VM.

malgré kubectl

proxy actif

Ces cinq problèmes partagent un pattern commun : chacun découle d'une hypothèse implicite sur la portée réseau ou système de fichiers (loopback conteneur vs hôte, résolution de chemin spécifique à l'outil Windows, détection de changements Kubernetes) qui n'est devenue visible qu'une fois la stack complète exercée de bout en bout — renforçant la valeur du système déployé et fonctionnel comme cible de validation réelle, plutôt que chaque composant testé en isolation.

Chapitre 5. Implémentation, tests et déploiement Plateforme AIOps

5.11 Sécurité

La sécurité est adressée à plusieurs couches, résumées dans le tableau 5.7.

Tableau 5.7 : Mesures de sécurité

Couche Mesure

Authentification JWT accès (15 min) + refresh (7 j), RBAC à trois niveaux

Webhooks HMAC-SHA256 avec comparaison en temps constant

Mots de passe Hachage Bcrypt (12 rounds de sel)

HTTP Helmet (CSP, HSTS, X-Frame-Options)

Rate limiting Store Redis par endpoint (webhook 30/min, API 100/min)

Secrets .env et values.secrets.yaml exclus du contrôle de version

Kubernetes Secrets Opaque injectés via Helm

5.12 Résultats

5.12.1 Métriques de performance

Tableau 5.8 : Résultats de performance

Indicateur Résultat

Temps d'analyse IA (webhook -> résultat) < 10 secondes

Précision de classification des types d'erreur 85% (cas testés)

Réduction du temps de diagnostic 30-90 min -> < 2 min

Temps de traitement BullMQ (sans LLM) < 1 seconde

Latence WebSocket (webhook -> tableau de bord) < 500 ms

5.12.2 Validation de bout en bout

Un test de bout en bout complet a été validé dans des conditions réelles : un git push déclenche un pipeline GitLab ; le webhook atteint le backend en moins d'une seconde ; un job BullMQ collecte les logs CI ; Groq effectue l'analyse ; le résultat est stocké dans MongoDB ; la base de connaissances et le score de santé sont mis à jour ; et le tableau de bord React se rafraîchit en temps réel via WebSocket — toute la chaîne se complétant en sept à dix secondes.

5.12.3 Fonctionnalités validées

Toutes les fonctionnalités ciblées ont été implémentées et validées : réception de webhook sécurisée, analyse de cause racine par IA, intégration GitLab/SonarQube/Trivy, base de connaissances auto-alimentée, score de santé par projet, cycle de vie complet des incidents avec post-mortems IA, commentaires automatiques de risque sur MR, rapports hebdomadaires, tableau de bord temps réel, sécurité JWT/RBAC, suite de 25 tests, builds Docker multi-étapes, déploiement Helm sur k3s, et pipeline Jenkins CI/CD complet.

5.13 Conclusion

Ce chapitre a présenté l'implémentation concrète, les tests et le déploiement de la plateforme, validant les objectifs définis au premier chapitre. Une ingénierie honnête exige cependant d'examiner là où la solution n'atteint pas encore un système de niveau production idéal — ce qui fait l'objet du chapitre suivant.

CHAPITRE 6

Discussion critique, limites et mitigations

6.1 Introduction

Un projet de fin d'études défendable ne doit pas seulement présenter ce qui fonctionne, mais aussi raisonner explicitement sur ce qui ne fonctionne pas encore, et pourquoi.

Ce chapitre anticipe les questions qu'un jury d'examen est le plus susceptible de soulever sur la plateforme décrite aux chapitres 3 et 4, organisées par catégorie, avec la mitigation déjà en place ou planifiée.

6.2 Limites méthodologiques

6.2.1 Précision de classification IA mesurée sur un échantillon limité

La précision de classification de 85% reportée dans le tableau 5.8 a été mesurée sur quelques dizaines d'échecs de pipelines réels observés pendant les sprints du projet, et non sur un benchmark étiqueté statistiquement représentatif. C'est une faiblesse méthodologique : le chiffre doit être lu comme un signal encourageant précoce, non comme un SLA de production validé. Un suivi rigoureux construirait un jeu de données étiqueté d'au moins quelques centaines d'échecs à travers plusieurs projets et rapporterait précision/rappel par `errorType` plutôt qu'une précision agrégée unique.

6.2.2 Absence d'étude d'utilisabilité formelle

L'utilisabilité du tableau de bord a été validée de manière informelle (usage propre de l'auteur, et bref retour de l'équipe hôte) plutôt que par un test d'utilisabilité structuré avec des ingénieurs cibles. Un questionnaire System Usability Scale (SUS) administré à l'unité DevOps renforcerait l'affirmation que l'outil réduit mesurément la charge cognitive, au-delà des mesures de temps du tableau 1.3.

6.3 Limites architecturales

Chapitre 6. Discussion critique, limites et mitigations Plateforme AIOps

6.3.1 Réplica unique, pas de haute disponibilité

Chaque Deployment dans le chart Helm actuel fonctionne avec un seul réplica. Le backend est cependant conçu sans état par construction — l'état de session réside dans les JWTs, pas en mémoire de processus, et l'état des jobs réside dans Redis/BullMQ — donc le passage à l'échelle horizontal à replicas: N est une modification d'une ligne dans `values.yaml` sans modification de code. Ce qui manque genuinely est un

HorizontalPodAutoscaler et un health-check d'équilibreur de charge configuré pour l'endpoint /api/health, deux ajouts simples différés pour des raisons de temps.

6.3.2 Instance MongoDB unique, sans jeu de réplicas ni politique de sauvegarde

MongoDB fonctionne comme un pod unique avec un seul PVC sans jeu de réplicas, ce qui signifie qu'une défaillance de nœud ou une corruption de PVC causerait une perte de données. La voie de mitigation est bien comprise (un jeu de réplicas à 3 nœuds via le chart Helm MongoDB Bitnami, plus un CronJob mongodump planifié vers un stockage objet), mais n'a pas été implémentée dans le budget temps du projet, car le cluster de lab n'a qu'un seul nœud VM.

6.3.3 Gestion des secrets sans coffre-fort

Les valeurs sensibles (secret JWT, token GitLab, clés API LLM) sont conservées dans un values.secrets.yaml exclu du dépôt git, injectées comme secrets Kubernetes Opaque (encodés en base64, non chiffrés au repos par défaut dans etcd). Un déploiement de production devrait utiliser un gestionnaire de secrets (HashiCorp Vault, ou Sealed Secrets / SOPS pour une alternative compatible GitOps) pour que les secrets soient chiffrés à la fois au repos et dans l'historique Git du dépôt de déploiement.

6.4 Confidentialité des données et dépendance au LLM tiers

Chaque appel LLM envoie un extrait de contexte (lignes de logs, messages de commits, descriptions d'issues SonarQube) à une API tierce (Groq, avec Claude comme secours). Pour les projets de lab utilisés durant ce PFE, c'est un compromis acceptable, mais ne le serait pas pour un projet client sous clauses strictes de résidence des données ou de confidentialité. Trois mitigations sont déjà partiellement en place ou planifiées :

(1) l'étape de corrélation/normalisation ne transmet que l'extrait pertinent du log d'un job en échec, limitant déjà l'exposition ; (2) le cache de la base de connaissances signifie qu'une erreur récurrente n'est diagnostiquée par le LLM qu'une seule fois, la solution mémorisée étant réutilisée ensuite sans que d'autres données quittent la plateforme ; (3) la conception dual-fournisseur (Groq primaire, Claude secours) pourrait être étendue avec une troisième option auto-hébergée (ex. un modèle Llama quantisé servi via Ollama dans le cluster) pour les clients nécessitant une inférence sur site, au prix de la qualité de classification.

Chapitre 6. Discussion critique, limites et mitigations Plateforme AIOps

6.5 Limites opérationnelles et de processus

- Déclenchement manuel du CI/CD : le pipeline Jenkins est actuellement démarré

via « Build Now » plutôt qu'un webhook GitHub automatique, car l'instance Jenkins de lab n'est pas exposée sur un endpoint public accessible depuis GitHub. Le Jenkinsfile et la logique du pipeline sont déjà prêts pour les webhooks ; seule la configuration du déclencheur manque.

- RBAC à granularité grossière : les trois rôles (admin/analyst/viewer) sont globaux, non limités par projet. Un client intégrant plusieurs équipes avec différentes visibilités de projets nécessiterait des permissions au niveau projet, non implémentées dans le modèle actuel.
- Couverture de tests automatisés limitée : comme discuté en section 5.6.2, seule la couche unitaire de la pyramide de tests est automatisée ; les tests d'intégration et end-to-end sont identifiés mais pas encore implémentés.
- Pas de supervision de la plateforme elle-même : la plateforme AIOps supervise les pipelines CI/CD clients, mais sa propre santé (latence API, profondeur de la file worker, taux d'erreur LLM) n'est pas encore exportée vers Prometheus/Grafana — une lacune honnête mais ironique pour un outil orienté observabilité.

6.6 Questions de jury anticipées

Le tableau 6.1 compile, sous forme de questions/réponses directes, les critiques les plus susceptibles d'être soulevées lors de la soutenance.

Tableau 6.1 : Questions de jury anticipées et réponses préparées

Question Réponse préparée

Pourquoi faire confiance à l'analyse Les types d'erreur sont trop variés pour un de cause racine d'un LLM plutôt ensemble exhaustif de règles ; le LLM est qu'à un moteur de règles contraint par un prompt structuré et sa déterministe ? sortie est toujours une suggestion examinée par un analyste, jamais une action automatique, et la base de connaissances capture les solutions validées pour réduire la dépendance au LLM avec le temps.

Que se passe-t-il si Groq est hors Le client Anthropic Claude est câblé comme service ? fallback automatique ; si les deux sont indisponibles, le job est relancé (backoff BullMQ) et se termine finalement comme une analyse échouée sans bloquer le pipeline lui-même.

Cette solution est-elle prête pour La couche applicative l'est ; la couche

la production ? infrastructure (réplica unique, instance Mongo unique, secrets sans coffre-fort) est de niveau lab et l'écart vers la production est explicitement cartographié dans ce chapitre, pas caché.

Le chiffre de précision de 85% Ce n'est pas un benchmark validé, seulement est-il fiable ? un signal précoce issu d'une revue manuelle des exécutions de projet ; la section 6.2.1 énonce cette limitation explicitement et propose un suivi par jeu de données étiqueté.

Pourquoi MongoDB plutôt que PostgreSQL ? La forme du document varie selon le type d'erreur et plusieurs champs sont toujours lus/écrits ensemble avec leur parent ; la justification est détaillée en section 4.7.2.

6.7 Conclusion

Aucune des limitations discutées dans ce chapitre n'invalidé la contribution principale — une plateforme opérationnelle de bout en bout, assistée par IA pour l'analyse CI/CD, déployée sur Kubernetes avec son propre pipeline CI/CD — mais les reconnaître précisément est ce qui transforme le projet d'une démonstration en un artefact d'ingénierie dont la prochaine itération est déjà cartographiée.

CHAPITRE 7

Gestion du projet

7.1 Introduction

Ce chapitre documente comment le projet a été planifié et suivi en tant que PFE solo avec contrainte de temps, complétant le découpage en sprints inspiré de Scrum déjà introduit en section 1.6 (figure 1.2), avec une structure de découpage du travail et un registre des risques.

7.2 Structure de découpage du travail (WBS)

La figure 7.1 décompose le projet en quatre lots de travail alignés sur les quatre chapitres d'implémentation de ce rapport, chacun subdivisé en livrables concrets produits.

Plateforme AIOps PFE

WT1 WT2 WT3 WT4

Tests & qualité Conteneurisation & CI/CD

Backend core & IA Tableau de bord frontend

Webhooks, Pages & Suites Vitest Docker,
queues, modèles composants (25 tests) Helm, k3s

Orchestrateur Gestion Validation Jenkins

IA, KB, score état & WebSocket manuelle e2e pipeline, GitHub
de santé

Figure 7.1 : Structure de découpage du travail (WBS)

7.3 Registre des risques

Le tableau 7.1 liste les risques techniques identifiés au début du projet, évalués sur une échelle probabilité/impact 1-3, avec la mitigation effectivement appliquée. Deux risques se sont matérialisés pendant le projet (marqués « survenu ») : la contention de ressources VM, observée quand GitLab CE, Jenkins et k3s tournaient simultanément sur la même VM 4-vCPU et causaient une charge moyenne de 25, correspondant exactement au log de débogage de la section 5.10.

Chapitre 7. Gestion du projet Plateforme AIOps

Tableau 7.1 : Registre des risques

Risque Prob. Impact Mitigation

Panne ou rate limiting 2 2 Fallback dual-fournisseur (Groq + Claude),
API LLM relances avec backoff

Contention VM 3 2 Images légères, étapes de pipeline
(GitLab+Jenkins+k3s) – séquentielles ; survenu (section 5.10)
survenu

Dérive du périmètre 3 3 Frontières de sprint strictes (figure 1.2),
(9 objectifs ambitieux, ordonnancement MVP-first
développeur solo)

Friction tooling 2 1 Contournement par chemin partagé pour
Windows/Linux (résolution l'authentification crumb ; survenu (section 5.10)
de chemins)

Perte de connaissance 1 2 Ce rapport écrit lui-même, plus documentation
entre sprints (développeur de code inline
solo, longues pauses)

7.4 Répartition de l'effort

Le tableau 7.2 donne une distribution approximative de l'effort à travers les activités
macro du projet, auto-reportée à la fin de chaque sprint.

Tableau 7.2 : Répartition approximative de l'effort

Activité Part de l'effort

Développement backend (API, files, orchestration IA) 30%

Développement frontend (tableau de bord React) 20%

Infrastructure (Vagrant, GitLab, Jenkins, k3s, Helm) 25%

Tests et débogage 15%

Documentation et ce rapport 10%

7.5 Conclusion

Ce chapitre complète le récit d'ingénierie du rapport avec la vue gestion de projet :
comment le périmètre a été découpé, quels risques ont été anticipés et lesquels se sont
matérialisés, et où l'effort est réellement allé. La conclusion générale suit.

Conclusion générale et perspectives

La plateforme AIOps développée dans ce projet de fin d'études répond à un problème concret auquel font face quotidiennement les équipes DevOps de Capgemini – Altran Telnnet Corporation Tunisie. Au-delà du simple diagnostic de pipeline, elle constitue un système d'intelligence opérationnelle complet : analyse de cause racine automatique en moins de dix secondes, mémoire organisationnelle via la base de connaissances auto-alimentée, pilotage continu de la qualité via les scores de santé, gestion structurée des incidents avec traçabilité et apprentissage, et prévention par l'analyse de risque des merge requests.

La réduction mesurée du temps de diagnostic — de 30-90 minutes à moins de deux minutes — représente un gain de productivité significatif, réduisant la charge cognitive de la gestion d'incidents et libérant les ingénieurs pour des tâches à plus forte valeur ajoutée. Entièrement conteneurisée et déployée sur Kubernetes avec un pipeline CI/CD Jenkins, la solution est extensible et prête pour un passage en production.

Sur le plan personnel, ce travail a consolidé des compétences en développement full-stack, architecture événementielle, intégration LLM et pratiques DevOps/Cloud.

Plusieurs directions futures se dégagent :

- alerting actif (email, Slack, PagerDuty) sur les incidents critiques ;
- ML prédictif estimant la probabilité d'échec d'un pipeline avant son exécution ;
- support d'autres plateformes CI/CD (GitHub Actions, Azure DevOps) ;
- export de métriques Prometheus pour des tableaux de bord Grafana ;
- isolation multi-tenant avec quotas par équipe.

Bibliographie

- [1] Gartner, AIOps (Artificial Intelligence for IT Operations) Platforms, IT Glossary.
- [2] Documentation GitLab, Webhooks et pipelines CI/CD, <https://docs.gitlab.com>.
- [3] SonarSource, Documentation SonarQube, <https://docs.sonarsource.com>.
- [4] Aqua Security, Trivy Vulnerability Scanner, <https://trivy.dev>.
- [5] BullMQ, Premium Message Queue for NodeJS, <https://docs.bullmq.io>.
- [6] Groq, Référence API GroqCloud, <https://console.groq.com/docs>.
- [7] Rancher, k3s – Lightweight Kubernetes, <https://k3s.io>.
- [8] The Helm Authors, Documentation Helm, <https://helm.sh/docs>.
- [9] Jenkins, Documentation Pipeline, <https://www.jenkins.io/doc>.
- [10] Meta, Documentation React, <https://react.dev>.

CHAPITRE A

Fichier Helm Values complet

Le listing A.1 reproduit le fichier values.yaml complet utilisé pour déployer la plateforme sur k3s (le fichier values.secrets.yaml complémentaire, contenant le secret JWT, le token GitLab et les clés API LLM, est exclu du dépôt git et intentionnellement non reproduit ici).

1 backend:

2 image:

3 repository: khalil512/aiops-backend

4 tag: latest

5 replicas: 1

6 port: 3001

7 resources:

8 requests: { cpu: 100m, memory: 256Mi }

9 limits: { cpu: 500m, memory: 512Mi }

11 frontend:

12 image:

13 repository: khalil512/aiops-frontend

14 tag: latest

15 replicas: 1

16 port: 80

17 resources:

18 requests: { cpu: 50m, memory: 64Mi }

19 limits: { cpu: 200m, memory: 128Mi }

21 mongodb:

22 image: mongo:4.4

23 persistence:

24 size: 2Gi

25 resources:

26 requests: { cpu: 100m, memory: 256Mi }

27 limits: { cpu: 500m, memory: 512Mi }

29 redis:

30 architecture: standalone

31 auth:

32 enabled: false

34 config:

35 nodeEnv: production

36 mongodbUri: mongodb://mongodb:27017/aiops

37 redisHost: redis-master

38 redisPort: "6379"

39 groqModel: llama-3.3-70b-versatile

41 ingress:

42 enabled: true

43 className: traefik

44 host: aiops.local

46 secrets: {} # surchargé par le values.secrets.yaml exclu du dépôt

Listing A.1 : Fichier helm/aiops/values.yaml complet

CHAPITRE B

Jenkinsfile complet

Le listing B.1 reproduit le pipeline déclaratif complet, dont seul un extrait condensé était présenté comme listing 5.8 au chapitre 5.

```
1 pipeline {
2   agent any
3   environment {
4     DOCKERHUB_TOKEN = credentials('dockerhub-token')
5     KUBECONFIG = '/var/jenkins_home/.kube/config'
6     SECRETS_FILE =
7       '/home/vagrant/aiops-secrets/values.secrets.yaml'
8   }
9   stages {
10    stage('Installer dépendances backend') {
11      steps { sh 'cd aiops-platform/backend && npm ci' }
12    }
13    stage('Tester backend') {
14      steps { sh 'cd aiops-platform/backend && npm test' }
15    }
16    stage('Installer dépendances frontend') {
17      steps { sh 'cd aiops-platform/frontend && npm ci' }
```



```
18 stage('Tester frontend') {
19 steps { sh 'cd aiops-platform/frontend && npm test' }
20 }
21 stage('Build image backend') {
22 steps { sh 'docker build -t khalil512/aiops-backend:latest
23 aiops-platform/backend' }
24 }
25 stage('Build image frontend') {
26 steps { sh 'docker build -t khalil512/aiops-frontend:latest
27 aiops-platform/frontend' }
28 }
29 stage('Pousser images') {
30 steps {
31 sh '''
32 echo "$DOCKERHUB_TOKEN" | docker login -u khalil512
--password-stdin
33 docker push khalil512/aiops-backend:latest
34 docker push khalil512/aiops-frontend:latest
35 '''
36 }
37 }
38 stage('Déployer sur k3s') {
```

```
39 steps {  
40 sh ""  
41 helm upgrade aiops aiops-platform/helm/aiops \  
42 -f aiops-platform/helm/aiops/values.yaml \  
43 -f "$SECRETS_FILE" \  
44 --wait --timeout 5m  
45 kubectl rollout restart deployment/aiops-backend  
deployment/aiops-frontend  
46 kubectl rollout status deployment/aiops-backend --timeout=120s  
47 kubectl rollout status deployment/aiops-frontend --timeout=120s  
48 ""  
49 }  
50 }  
51 }  
52 }
```

Listing B.1 : Fichier aiops-platform/Jenkinsfile complet

CHAPITRE C

Extraits de code supplémentaires

C.1 Service de score de santé : correspondance grade complète

```
1 function scoreToGrade(score) {  
2   if (score >= 90) return 'A';  
3   if (score >= 75) return 'B';  
4   if (score >= 60) return 'C';  
5   if (score >= 45) return 'D';  
6   return 'F';  
7 }
```

Listing C.1 : scoreToGrade (complet)

C.2 Service base de connaissances : génération de signature

```
1 const crypto = require('crypto');  
3 function generateSignature(errorType, rootCause) {  
4   const normalized = `${errorType}:${rootCause.slice(0, 100)}`  
5   .toLowerCase()  
6   .trim();  
7   return crypto.createHash('md5').update(normalized).digest('hex');  
8 }
```

Listing C.2 : generateSignature (complet)

C.3 Frontend : client Axios avec rafraîchissement automatique du token

```
1 apiClient.interceptors.response.use(  
2 (response) => response,  
3 async (error) => {  
4 if (error.response?.status === 401 && !error.config._retried) {  
5 error.config._retried = true;  
6 const { accessToken } = await authApi.refresh();  
7 store.dispatch(setAccessToken(accessToken));  
8 error.config.headers.Authorization = `Bearer ${accessToken}`;  
9 return apiClient(error.config);  
10 }  
11 return Promise.reject(error);  
12 }  
13 );
```

Listing C.3 : Intercepteur Axios rafraîchissant un access token expiré

CHAPITRE D

Glossaire

Analyse de cause racine

Le processus d'identification de la cause technique sous-jacente d'un échec observé, par opposition à la simple description de ses symptômes.

Signature de base de connaissances

Un hash MD5 déterministe du type d'erreur et des 100 premiers caractères de la cause racine normalisée, utilisé comme clé de cache pour les solutions déjà diagnostiquées.

Score de santé Un indicateur composite de 0 à 100 calculé à partir de six métriques opérationnelles pondérées, converti en grade (A à F) par projet supervisé.

Cache hit / miss Dans le contexte de cette plateforme, si une signature d'erreur donnée est déjà présente dans la base de connaissances (hit, solution réutilisée) ou non (miss, le LLM est invoqué).

Webhook Un callback HTTP qu'un système externe (ici, GitLab) émet automatiquement lorsqu'un événement se produit (fin d'un pipeline, ouverture d'une merge request).

BullMQ Une bibliothèque de file de messages Redis pour Node.js, utilisée ici pour découpler l'ingestion des webhooks de l'analyse LLM plus lente.

Chart Helm Une description packagée et paramétrée d'un ensemble de ressources Kubernetes, installée et mise à jour comme une seule unité versionnée.

k3s Une distribution Kubernetes certifiée et légère, adaptée aux environnements contraints en ressources ou de lab/edge.

MTTR Mean Time To Resolve : la durée moyenne entre la détection d'un incident et sa résolution.

LLM Large Language Model : un grand modèle de langage entraîné sur d'importants corpus textuels, capable de génération et de raisonnement en langage naturel.